ğrencibilgiLab 02 – Hafta 2İteratörler ve STL Algoritmaları

Lab Hakkında

Bu laboratuvar çalışmasında STL iteratörlerini, temel ve ileri algoritmaları ve lambda ifadelerini uygulamalı olarak kullanacaksınız. Her çalışma bölümünde önce örnek kodu inceleyip çalıştırın, ardından görevleri tamamlayın.

Süre: 2 saat **Derleme:** `g++ -std=c++20 -o program dosya.cpp`

1 Çalışma 1: İteratör Kullanımı (25 dk)

Ön Bilgi

İteratörler kapsayıcılardaki elemanlara sıralı erişim sağlar. `begin()` ilk elemana, `end()` son elemanın bir sonrasına işaret eder. C++17 ile **structured bindings** kullanarak `map` elemanlarına zarif biçimde erişebilirsiniz.

Aşağıdaki başlangıç kodunu `çalışma1.cpp` dosyasına yazın ve derleyin:

```cpp
#include <iostream>
#include <map>
#include <string>
#include <iterator>

int main() {
    std::map<int, std::string> öğrenciler = {
        {2001, "Ali Yilmaz"},
        {2002, "Ayse Kaya"},
        {2003, "Mehmet Demir"},
        {2004, "Zeynep Can"},
        {2005, "Can Ozturk"}
    };

    // C++17 structured bindings ile dolasma
    std::cout << "--- Öğrenci Listesi ---\n";
    for (const auto& [no, isim] : öğrenciler) {
        std::cout << no << ": " << isim << '\n';
    }

    // Iterator ile dolasma
    std::cout << "\n--- Iterator ile ---\n";
    for (auto it = öğrenciler.begin();
         it != öğrenciler.end(); ++it) {
        std::cout << it->first << ": "
                  << it->second << '\n';
    }

    return 0;
}
```

`std::distance` ve `std::advance`

`std::distance(first, last)` iki iteratör arası mesafeyi verir. `std::advance(it, n)` iteratörü `n` adım ilerletir. Bidirectional iteratörlerde negatif `n` ile geri gidilebilir.

Görev 1.1 – Ters İteratör ile Dolaşma (5 dk)

`rbegin()` ve `rend()` kullanarak öğrenci listesini **tersten** (büyük numaradan küçüğe) yazdırın.
Beklenen çıktı: 2005'ten 2001'e doğru listeleme.

Görev 1.2 – distance ve advance (10 dk)

a) `std::distance(öğrenciler.begin(), öğrenciler.end())` çağrısı ile eleman sayısını hesaplayın.

b) `std::advance` ile `begin()` iteratörünü 2 adım ilerletin ve o öğrenciyi yazdırın. Hangi öğrenci çıktı?

c) `std::next(it, n)` ile `std::advance(it, n)` arasındaki farkı test edin. (`next` yeni iteratör döner, `advance` mevcut iteratörü değiştirir.)

Görev 1.3 – const_iterator ve Güvenlik (10 dk)

a) `cbegin()` ve `cend()` kullanarak listeyi dolaşın.

b) `const_iterator` üzerinden bir öğrencinin ismini değiştirmeye çalışın. Ne olur?

c) Bir `for` döngüsünde `auto&` yerine `const auto&` kullanmanın neden önemli olduğunu bir yorum satırı ile açıklayın.

Dikkat

`map` iteratörü `Bidirectional` kategorisindedir. Dolayısıyla `it + 3` yazamazsınız (bu sadece Random Access iteratörlerde çalışır). Bunun yerine `std::advance(it, 3)` veya `std::next(it, 3)` kullanın.

Görev 1.4 – Akış İteratörleri (10 dk)

`ostream_iterator` kullanarak öğrenci numaralarını ekrana yazdırın:

```
#include <iterator>
// map'ten numaralari vector'e aktar
std::vector<int> numaralar;
for (const auto& [no, isim] : öğrenciler) {
    numaralar.push_back(no);
}
// ostream_iterator ile yazdir
std::copy(numaralar.begin(), numaralar.end(),
    std::ostream_iterator<int>(std::cout, ", "));
```

a) Ayırıcıyı `", "` yerine `" | "` olarak değiştirin.

b) Aynı tekniği öğrenci isimlerini yazdırmak için uygulayın (`ostream_iterator<std::string>`).

2 Çalışma 2: Sıralama ve Arama (20 dk)

Ön Bilgi

STL, farklı sıralama ihtiyaçları için `sort`, `stable_sort` ve `partial_sort` algoritmaları sunar. Arama için `find`, `find_if` ve sıralı veride `binary_search` kullanılır.

Aşağıdaki kodu `çalışma2.cpp` dosyasına yazın:

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>

struct Öğrenci {
    int numara;
    std::string isim;
    double gpa;
};

int main() {
    std::vector<Öğrenci> liste = {
        {2001, "Ali Yilmaz", 3.45},
        {2002, "Ayse Kaya", 3.80},
        {2003, "Mehmet Demir", 2.90},
        {2004, "Zeynep Can", 3.60},
        {2005, "Can Ozturk", 2.10},
        {2006, "Elif Aksoy", 3.95}
    };

    // GPA'ya gore sırala (buyukten kucuge)
    std::sort(liste.begin(), liste.end(),
        [](const Öğrenci& a, const Öğrenci& b) {
            return a.gpa > b.gpa;
        });

    std::cout << "--- GPA Sıralamasi ---\n";
    for (const auto& o : liste) {
        std::cout << o.isim << " - GPA: " << o.gpa << '\n';
    }

    return 0;
}
```

Görev 2.1 – find_if ile Arama (5 dk)

`std::find_if` kullanarak GPA'sı 3.5'in üzerinde olan **ilk** öğrenciyi bulun ve yazdırın.

```cpp
auto it = std::find_if(liste.begin(), liste.end(),
    [](const Öğrenci& o) { return o.gpa > 3.5; });
```

Görev 2.2 – binary_search ile Hızlı Arama (10 dk)

a) Listeyi **numaraya göre** küçükten büyüğe sıralayın.

b) `std::binary_search` ile 2004 numaralı öğrencinin varlığını kontrol edin.

c) `std::lower_bound` ile 2004 numaralı öğrencinin iteratörünü bulun ve bilgilerini yazdırın.

Soru: `binary_search` neden sadece `bool` döner ve iteratör dönmez? Konumu bulmak için ne kullanırsınız?

Görev 2.3 – partial_sort ve count_if (5 dk)

a) `std::partial_sort` ile sadece en yüksek GPA'lı 3 öğrenciyi sıralayın.

b) `std::count_if` ile GPA'sı 3.0'ın üzerindeki öğrenci sayısını bulun.

3 Çalışma 3: Transform ve Filtreleme (25 dk)

Ön Bilgi

`std::transform` her elemana bir dönüşüm uygular. `std::copy_if` koşulu sağlayan elemanları kopyalar. **Remove-erase idiomu** C++17'ye kadar standart silme yöntemiydi; C++20 ile `std::erase` ve `std::erase_if` kullanılır.

Aşağıdaki kodu `çalışma3.cpp` dosyasına yazın:

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
#include <string>

int main() {
    std::vector<int> notlar = {85, 42, 95, 60, 78, 92, 55, 88};

    // transform: not -> harf notu dönüşümü
    std::vector<std::string> harfler(notlar.size());
    std::transform(notlar.begin(), notlar.end(), harfler.begin(),
        [](int n) -> std::string {
            if (n >= 90) return "AA";
            if (n >= 80) return "BA";
            if (n >= 70) return "BB";
            if (n >= 60) return "CB";
            return "FF";
        });

    for (size_t i = 0; i < notlar.size(); ++i) {
        std::cout << notlar[i] << " -> " << harfler[i] << '\n';
    }

    return 0;
}
```

Görev 3.1 – copy_if ile Filtreleme (5 dk)

`std::copy_if` ve `std::back_inserter` kullanarak notları 70 ve üstü olan öğrencileri yeni bir `vector`'e kopyalayın. Kopyalanan eleman sayısını yazdırın.

Görev 3.2 – Remove-Erase İdiomu (10 dk)

a) **C++17 yöntemi:** Remove-erase idiomu ile 60'ın altındaki notları silin:

```cpp
notlar.erase(
    std::remove_if(notlar.begin(), notlar.end(),
        [](int n) { return n < 60; }),
    notlar.end());
```

b) Silme öncesi ve sonrası `size()` değerini yazdırın.

c) **Deney:** Sadece `std::remove_if` çağırıp `erase` çağırmazsanız `size()` değişir mi? Neden?

Görev 3.3 – C++20 erase_if (5 dk)

Aynı işlemi C++20 `std::erase_if` ile tek satırda yapın:

```
std::erase_if(notlar, [](int n) { return n < 60; });
```

Soru: C++20 yaklaşımının avantajları nelerdir?

Görev 3.4 – transform ile Eğri Uygulama (5 dk)

Tüm notlara 10 puan ekleyen (ancak 100'ü geçmeyen) bir `transform` çağrısı yazın:

```
std::transform(notlar.begin(), notlar.end(), notlar.begin(),
    [](int n) { return std::min(n + 10, 100); });
```

`transform` Yerinde Dönüşüm

`transform`'da kaynak ve hedef aynı aralık olabilir. Bu durumda her eleman yerinde dönüştürülür (in-place transformation).

Görev 3.5 – fill

a) `std::fill` ile 8 elemanlık bir `vector<int>`'i sıfırlayın.

b) `std::generate` ile aynı vektörü öğrenci numaraları ile doldurun (2001'den başlayarak artan):

```
int numara = 2000;
std::generate(v.begin(), v.end(),
    [&numara]() { return ++numara; });
```

c) `std::for_each` ile her elemana 5 puan ekleyin (referans yakalama ile).

4 Çalışma 4: Lambda ve İleri Algoritmalar (30 dk)

Ön Bilgi

Lambda ifadeleri `[yakalama](parametreler){ gövde }` söz dizimi ile yerinde fonksiyon nesneleri oluşturur. Yakalama modu ile dış kapsamdaki değişkenlere erişebilirsiniz. `std::accumulate` ve `std::partition` ileri düzey algoritmalardır.

Aşağıdaki kodu `çalışma4.cpp` dosyasına yazın:

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
#include <functional>
#include <string>

struct Öğrenci {
    std::string isim;
    double gpa;
};

int main() {
    std::vector<Öğrenci> liste = {
        {"Ali", 3.45}, {"Ayse", 3.80}, {"Mehmet", 2.90},
        {"Zeynep", 3.60}, {"Can", 2.10}, {"Elif", 3.95}
    };

    // Lambda ile GPA esik degeri disaridan yakalama
    double esik = 3.0;
    auto basarili_mi = [esik](const Öğrenci& o) {
        return o.gpa >= esik;
    };

    int basarili_sayisi = std::count_if(
        liste.begin(), liste.end(), basarili_mi);
    std::cout << "GPA >= " << esik << ": "
              << basarili_sayisi << " öğrenci\n";

    return 0;
}
```

Görev 4.1 – Yakalama Modları (10 dk)

a) **Değerle yakalama:** `esik` değişkenini lambda içinde değiştirmeye çalışın. Ne olur? `mutable` ekleyerek tekrar deneyin.

b) **Referansla yakalama:** `[&esik]` ile yakalayın ve lambda içinde `esik = 3.5;` yazın. Lambda çağrısından sonra `esik` değeri ne olur?

c) Bir sayaç lambdası yazın: Her çağrıda değeri 1 artan bir lambda oluşturun (`mutable` kullanarak).

Görev 4.2 – accumulate ile GPA Hesaplama (5 dk)

`std::accumulate` kullanarak tüm öğrencilerin GPA ortalamasını hesaplayın:

```
double toplam = std::accumulate(liste.begin(), liste.end(),
    0.0, [](double t, const Öğrenci& o) {
        return t + o.gpa;
    });
double ortalama = toplam / liste.size();
```

Dikkat: Başlangıç değeri olarak `0.0` (double) yazın, `0` (int) değil!

Görev 4.3 – inner_product ile Ağırlıklı GNO (10 dk)

OBYS'de ağırlıklı GNO hesaplamak için `std::inner_product` kullanın:

```
#include <numeric>
std::vector<int> krediler = {3, 4, 3, 2};
std::vector<double> notlar = {3.5, 2.8, 3.9, 4.0};

double agirlikli = std::inner_product(
    krediler.begin(), krediler.end(),
    notlar.begin(), 0.0);
int toplam_kredi = std::accumulate(
    krediler.begin(), krediler.end(), 0);
double gno = agirlikli / toplam_kredi;
```

a) Kodu çalıştırın ve GNO'yu yazdırın.

b) Bir öğrencinin daha notlarını ekleyin ve GNO'yu yeniden hesaplayın.

c) `std::partial_sum` ile kümülatif kredi toplamını hesaplayın.

Görev 4.4 – partition ile Gruplandırma (10 dk)

a) `std::partition` ile öğrencileri GPA $\geq$ 3.0 ve GPA $<$ 3.0 olarak ikiye ayırın.

b) Dönen iteratörü kullanarak her iki grubu ayrı ayrı yazdırın.

c) `std::stable_partition` ile tekrar yapın ve sıra farkını gözlemleyin.

Görev 4.5 – Bonus: Birleştirilebilir Filtreler ve IIFE (10 dk)

`std::function` kullanarak farklı filtreleri birleştirin:

```
using Filtre = std::function<bool(const Öğrenci&)>;

Filtre gpa_filtre = [](const auto& o) { return o.gpa >= 3.0; };
Filtre isim_filtre = [](const auto& o) { return o.isim[0] == 'A'; };

// VE birlesimi
auto ve = [=](const Öğrenci& o) {
    return gpa_filtre(o) && isim_filtre(o);
};
```

Bu birleşik filtreyi `copy_if` ile kullanarak sonuçları yazdırın.
Bonus 2 – IIFE: Aşağıdaki IIFE (hemen çağrılan lambda) desenini kullanarak bir `const` not tablosu oluşturun:

```
const auto harf_tablosu = []() {
    std::map<std::string, double> t;
    t["AA"] = 4.0; t["BA"] = 3.5; t["BB"] = 3.0;
    t["CB"] = 2.5; t["CC"] = 2.0; t["FF"] = 0.0;
    return t;
}(); // () -> hemen cagir!
```

Dikkat: Lambda Yaşam Süresi

Referansla yakalanan değişken lambda'dan önce yok edilirse **tanımsız davranış** oluşur! Uzun ömürlü lambdalarda değerle yakalama tercih edin.

Teslim Bilgileri

Teslim edilecekler:

1. `çalışma1.cpp` – İteratör görevleri (Görev 1.1–1.4)
2. `çalışma2.cpp` – Sıralama ve arama (Görev 2.1–2.3)
3. `çalışma3.cpp` – Transform ve filtreleme (Görev 3.1–3.5)
4. `çalışma4.cpp` – Lambda ve ileri algoritmalar (Görev 4.1–4.4)
5. *Bonus:* Birleştirilebilir filtreler ve IIFE (Görev 4.5)

Puanlama: Çalışma 1: 25p Çalışma 2: 20p Çalışma 3: 25p Çalışma 4: 20p Bonus: 10p